

Docker Messages App

Setup & Run Guide: every command, step by step

MySQL 8.4 · Spring Boot 3 / Java 21 · Next.js · Docker Compose

Service	Folder	Technology	Port
db	(official image)	MySQL 8.4	3306 (internal only)
backend	DockerBackend/	Spring Boot 3, Java 21, Maven	8080
frontend	DockerFe/	Next.js, TypeScript	3000

1. Prerequisites

You only need **Docker Desktop** to run the project. Java, Maven and Node are *not* required, because the Docker images build everything inside containers.

1.1 Install Docker Desktop

Download it from docker.com/products/docker-desktop (Mac, Windows or Linux), install it and **start it**. Wait until the whale icon says Docker is running.

1.2 Check that Docker works

```
docker --version
docker compose version
docker run --rm hello-world
```

All three commands should succeed. `hello-world` prints a "Hello from Docker!" message.

1.3 Make sure ports 3000 and 8080 are free

The app uses port **3000** (frontend) and **8080** (backend) on your computer. If another program is using either port, Docker can't start the container.

```
# macOS / Linux: no output means the port is free
lsof -i :3000
lsof -i :8080

# Windows (PowerShell / CMD)
netstat -ano | findstr :3000
netstat -ano | findstr :8080
```

Watch out: A common culprit is a forgotten `npm run dev` on port 3000. Stop it (Ctrl+C in its terminal), or see section 8 to change the port.

2. Project structure

```
root folder/
|-- DockerBackend/      Spring Boot REST API
|   |-- Dockerfile      multi-stage: Maven+JDK build -> slim JRE runtime
|   |-- .dockerignore
|   |-- pom.xml
|   |-- src/main/...    ApiController, Message, MessageRepository, application.yml
|-- DockerFe/           Next.js app (App Router, TypeScript)
|   |-- Dockerfile      multi-stage: deps -> builder -> runner
|   |-- .dockerignore
|   |-- next.config.ts  output: "standalone" + /api/* rewrite to the backend
|   |-- app/page.tsx    health status, message list, add-message form
|-- docker-compose.yml  db + backend + frontend, network, volume
|-- .env.example        database name, user and passwords
|-- README.md           the full lesson for students
```

API endpoints

Method	URL	What it does
GET	/api/health	Returns {"status": "UP"}
GET	/api/messages	Lists all messages stored in MySQL
POST	/api/messages	Adds a message. Body: {"text": "..."}
GET	/actuator/health	Spring Actuator health check (also checks the DB connection)

3. Run everything with Docker Compose (recommended)

Step 1: Open a terminal in the project root

```
cd "path/to/root folder"
```

Note: The folder name contains a space, so keep the quotes.

Step 2: Create your .env file

```
# macOS / Linux
cp .env.example .env

# Windows (CMD)
copy .env.example .env
```

The .env file holds the database name, user and passwords. Compose reads it automatically. You can keep the default values. (If .env is missing, the same defaults are used anyway.)

```
MYSQL_ROOT_PASSWORD=root_password
DB_NAME=messages_db
DB_USER=app
DB_PASSWORD=app_password
```

Step 3: Build and start all services

```
docker compose up --build
```

The first run takes several minutes, because Docker downloads the base images (MySQL, Maven, Java, Node) plus all Maven and npm dependencies. Later runs are much faster thanks to layer caching. Compose starts the services **in order**:

Order	Service	Starts when...
1	db	immediately; becomes healthy when MySQL accepts connections
2	backend	db is healthy; becomes healthy when /actuator/health is UP
3	frontend	backend is healthy

To run it in the background instead (so you get your terminal back), add **-d**:

```
docker compose up --build -d
```

Step 4: Open the app

What	URL
Frontend (the app)	http://localhost:3000
Backend health	http://localhost:8080/api/health
Backend messages	http://localhost:8080/api/messages
Actuator health	http://localhost:8080/actuator/health

On the page you should see **Backend status: UP**. Type a message and click **Add**, and it appears in the list.

4. Check that everything works

4.1 All services healthy?

```
docker compose ps
```

Expected: **db** and **backend** show (healthy), and **frontend** shows Up.

4.2 Test the API from the terminal

```
# directly against the backend (port 8080)
curl http://localhost:8080/api/health

# through the frontend's /api rewrite (port 3000), the same path the browser uses
curl http://localhost:3000/api/health

# add a message
curl -X POST http://localhost:3000/api/messages \
  -H "Content-Type: application/json" \
  -d '{"text":"Hello Docker"}'

# list messages
curl http://localhost:3000/api/messages
```

Note: On Windows PowerShell, use `curl.exe` instead of `curl`. The single-quoted JSON also works best in Git Bash or WSL.

4.3 Look inside the database

```
docker compose exec db mysql -u app -papp_password messages_db
mysql> SELECT * FROM messages;
mysql> exit
```

5. Everyday Docker Compose commands

Command	What it does
<code>docker compose up --build</code>	Build images (if changed) and start all services
<code>docker compose up --build -d</code>	Same, in the background (detached)
<code>docker compose ps</code>	List services, status, health and ports
<code>docker compose logs -f</code>	Follow the logs of all services (Ctrl+C stops following)
<code>docker compose logs -f backend</code>	Follow the logs of one service
<code>docker compose exec backend sh</code>	Open a shell inside the backend container (exit to leave)
<code>docker compose exec frontend sh</code>	Open a shell inside the frontend container
<code>docker compose restart backend</code>	Restart one service
<code>docker compose stop</code>	Stop the containers, but don't remove them
<code>docker compose down</code>	Stop and remove containers + network. Data is kept.
<code>docker compose down -v</code>	Same, plus delete the database volume (empty DB next time)
<code>docker compose build --no-cache</code>	Rebuild the images from scratch, ignoring the cache

After you change code

`docker compose up` on its own reuses the existing images. Rebuild them after every code change:

```
docker compose up --build      # rebuild changed services and restart them
docker compose up --build backend # rebuild/restart only the backend
```

6. Run each service manually (without Compose)

This is the "hard way" from Lessons 1 and 2 in the README. It shows exactly what Compose automates for you.

6.1 Create a network

```
docker network create lesson-net
```

6.2 Start MySQL

```
docker run -d --name db --network lesson-net \
-e MYSQL_ROOT_PASSWORD=root_password \
-e MYSQL_DATABASE=messages_db \
-e MYSQL_USER=app \
-e MYSQL_PASSWORD=app_password \
mysql:8.4
```

```
docker logs -f db      # wait for "ready for connections", then Ctrl+C
```

6.3 Build and run the backend

```
docker build -t docker-backend ./DockerBackend

docker run -d --name backend --network lesson-net \
  -p 8080:8080 \
  -e DB_HOST=db \
  docker-backend

curl http://localhost:8080/api/health
```

6.4 Build and run the frontend

```
docker build -t docker-frontend ./DockerFe

docker run -d --name frontend --network lesson-net \
  -p 3000:3000 \
  docker-frontend
```

Open <http://localhost:3000>. The frontend finds the backend because the container is named **backend** and both are on **lesson-net**.

6.5 Clean up the manual setup

```
docker rm -f frontend backend db
docker network rm lesson-net
```

Watch out: Remove the manual containers **before** running `docker compose up`. Otherwise ports 3000 and 8080 are already taken.

Note: Windows CMD: replace the line-continuation `\` with `^`, or put each command on one line. In PowerShell, use a backtick ```.

7. Run without Docker (local development)

This is optional. It needs Java 21, Node 20+ and a MySQL server on localhost:3306 with the same database, user and password as above. The easiest way to get that MySQL is Docker itself:

```
docker run -d --name local-mysql -p 3306:3306 \
  -e MYSQL_ROOT_PASSWORD=root_password -e MYSQL_DATABASE=messages_db \
  -e MYSQL_USER=app -e MYSQL_PASSWORD=app_password mysql:8.4
```

Backend

```
cd DockerBackend
./mvnw spring-boot:run           # Windows: mvnw.cmd spring-boot:run
```

Frontend (in a second terminal)

```
cd DockerFe
npm install
npm run dev # proxies /api/* to http://localhost:8080 by default
```

8. Troubleshooting

Problem: "port is already allocated" / "address already in use"

Another program is using port 3000 or 8080. Find it and stop it:

```
lsof -i :3000 # macOS/Linux: shows the PID
kill <PID>
netstat -ano | findstr :3000 # Windows: PID is the last column
taskkill /PID <PID> /F
```

Or change the **left** side of the port mapping in docker-compose.yml, for example "4000:3000", and open <http://localhost:4000>.

Problem: "Cannot connect to the Docker daemon"

Docker Desktop isn't running. Start it, wait until it's ready, and try again.

Problem: My code change doesn't show up

The old image is still being used. Rebuild:

```
docker compose up --build
docker compose build --no-cache && docker compose up # if still stale
```

Problem: Changed the password in .env, now the backend can't connect

MySQL only creates the user on its **first** start with an empty volume. The old password still lives in the volume. Reset it (this deletes all messages):

```
docker compose down -v
docker compose up --build
```

Problem: Backend keeps restarting / frontend never starts

Look at the health status and the logs. The frontend waits until the backend is healthy, and the backend waits for the db.

```
docker compose ps
docker compose logs backend
docker compose logs db
```

Problem: Connection refused when using "localhost" inside a container

Inside a container, localhost means **the container itself**. Use the service name instead: db or backend.

Problem: Frontend shows "Backend status: DOWN"

Check that the backend is healthy (`docker compose ps`). If you changed `BACKEND_URL`, rebuild the frontend image. Next.js fixes the rewrite target at **build** time.

```
docker compose up --build frontend
```

Problem: The first build is very slow

That's normal. Docker is downloading base images and dependencies. Later builds reuse the cached layers.

9. Complete cleanup

```
docker compose down -v          # containers, network and DB volume
docker rmi rootfolder-backend rootfolder-frontend docker-backend docker-frontend
docker system prune             # optional: remove ALL unused Docker data
```

Note: Compose names images after the folder (for example `rootfolder-backend`). Run `docker images` to see the exact names on your machine. Ignore "No such image" errors for images you never built.

10. Quick reference: the whole flow

```
# 1. Check Docker
docker --version
docker compose version

# 2. Go to the project and create .env
cd "path/to/root folder"
cp .env.example .env

# 3. Build and start
docker compose up --build -d

# 4. Check
docker compose ps
curl http://localhost:3000/api/health
open http://localhost:3000          # or paste the URL into your browser

# 5. Watch logs
docker compose logs -f

# 6. After code changes
docker compose up --build -d

# 7. Stop (keep data) / stop and wipe data
docker compose down
docker compose down -v
```